

Esercizio A1 – Complessità – Punti ≥ 8

Data la forma della ricorrenza, è possibile utilizzare il Master Theorem, versione base.

- Per $a = 2$, abbiamo $\alpha = \log_a 2a = \log_2 4 = 2$, $\beta = 2$; siamo nel secondo caso, $T(n) = \Theta(n^2 \log n)$.
- Per $a = 3$, abbiamo $\alpha = \log_a 2a = \log_3 6 < 2$, $\beta = 3$; siamo nel terzo caso, $T(n) = \Theta(n^3)$.
- In generale, è possibile dimostrare che $\log_a 2a = \log_a a + \log_a 2 = 1 + \log_a 2 < 2$ per tutti i valori $a \geq 3$; siamo nel terzo caso e si ottiene $T(n) = \Theta(n^a)$.

Esercizio A2 – Somma ramificata – Punti ≥ 10

L'algoritmo proposto utilizza una visita in profondità (*depth-first search*, *DFS*) per calcolare la somma dei valori dei nodi che si trovano ad una distanza al più k dal nodo t . Questa distanza viene misurata in termini di passi lungo gli archi dell'albero.

```
int branchSum(TREE t, int k)
```

```
    return bsRec(t, k, nil)
```

```
int bsRec(TREE curr, int k, TREE prev)
```

```
    if curr == nil then
```

```
        return 0
```

```
    else
```

```
        int res = curr.value
```

```
% Inizializza il risultato con il valore del nodo corrente curr
```

```
        if k > 0 then
```

```
            if curr.left() != prev then
```

```
                res = res + bsRec(curr.left(), k - 1, curr)
```

```
            if curr.right() != prev then
```

```
                res = res + bsRec(curr.right(), k - 1, curr)
```

```
            if curr.parent() != prev then
```

```
                res = res + bsRec(curr.parent(), k - 1, curr)
```

```
        return res
```

La funzione `branchSum()` utilizza una funzione ricorsiva ausiliaria `bsRec`, che visita l'albero trattandolo come un grafo. Ogni nodo può avere fino a tre vicini: il figlio sinistro, quello destro e il genitore.

La funzione ricorsiva `bsRec()` riceve tre parametri:

- il nodo corrente *curr*,
- il numero massimo di passi residui *k*,
- il nodo precedente *prev*, che impedisce di ritornare indietro lungo il percorso già visitato (per evitare cicli e doppie conteggiature).

Nel caso il nodo considerato non sia **nil** (nel qual caso restituisce 0), il comportamento della funzione è il seguente:

1. Aggiunge il valore del nodo corrente *curr.value* alla somma parziale *res*.
2. Se $k > 0$ (ovvero se ci sono ancora passi disponibili):
 - chiama ricorsivamente `bsRec` sul figlio sinistro, a condizione che non sia il nodo precedente;
 - chiama ricorsivamente `bsRec` sul figlio destro, a condizione che non sia il nodo precedente;
 - chiama ricorsivamente `bsRec` sul nodo genitore, a condizione che non sia il nodo precedente.

In tutti e tre i casi il numero di passi disponibili *k* viene diminuito di uno, e il valore di ritorno delle funzioni viene sommato alla variabile *res*.

3. Restituisce la somma totale *res* accumulata fino a quel punto.

La correttezza dell'algoritmo si basa sul fatto che:

- Ogni nodo visitato viene aggiunto esattamente una volta alla somma totale, grazie al controllo esplicito che impedisce di ritornare al nodo precedente (*prev*). Poiché in un albero esiste uno e un solo cammino per andare da un nodo t a un qualunque altro nodo v , non è necessario un vettore *visited*.
- L'albero viene esplorato completamente fino alla distanza k da t , considerando tutti i percorsi possibili (verso i figli o il genitore).
- La somma include correttamente t stesso, anche nel caso $k = 0$, poiché il valore di t viene aggiunto all'inizio della chiamata ricorsiva.

Nel caso peggiore, l'algoritmo visita l'intero albero, quindi la complessità è $O(n)$, dove n è il numero di nodi dell'albero. Ad essere più precisi, a parte la prima chiamata sul nodo t , dove potenzialmente vengono visitati tre nodi vicini, in tutte le chiamate successive vengono visitati al più due nodi (escludendo quello da cui si proviene), quindi il numero di nodi visitati è limitato superiormente da $3 \cdot 2^{k-1} = O(2^k)$. Entrambi i limiti sono validi e quindi potremmo scrivere $O(\min\{n, 2^k\})$.

Esercizio A3 – Circular MaxSum – Punti ≥ 12

Una nota prima di iniziare: come nella Somma Massimale vista a lezione, stiamo considerando la possibilità che il vettore sia composto da tutti numeri negativi, nel qual caso il sottovettore massimale è quello vuoto di somma zero.

Soluzione $\Theta(n^2)$ Una soluzione banale consiste nel modificare la versione $\Theta(n^2)$ dell'algoritmo della somma massimale, in modo che consideri tutti i possibili sottovettori che partono da ogni indice i e hanno dimensione compresa fra 1 e n .

```

int circularMaxSum(int[] A, int n)
{
    int maxSoFar = 0
    for start = 1 to n do
        int sumSoFar = 0                                % Somma da start a start + j
        for j = 0 to n - 1 do
            sumSoFar = sumSoFar + A[(start + j) mod n]
            maxSoFar = max(maxSoFar, sumSoFar)
        return maxSoFar
}

```

Soluzione $\Theta(n)$ La procedura `circularMaxSum()` calcola la *somma massimale* di un sottovettore contiguo in un vettore circolare A di dimensione n . Il problema viene risolto considerando due casi principali:

1. **Sottovettore non circolare:** si identifica il sottovettore di somma massimale utilizzando l'*algoritmo di Kadane*, che restituisce la somma massima tra i sottovettori lineari (senza considerare la circolarità del vettore).
2. **Sottovettore circolare:** si considerano tutti i sottovettori circolari, ossia quelli che iniziano in una posizione *Start* e terminano in una posizione *End* con $End < Start$. In questi sottovettori, gli elementi nell'intervallo $[End + 1, Start - 1]$ non sono inclusi nella somma. Pertanto, il sottovettore circolare con somma massimale è quello in cui l'intervallo escluso $[End + 1, Start - 1]$ ha la somma *minimale*. Per identificare questo sottovettore, utilizziamo una variante dell'algoritmo di Kadane che calcola il sottovettore contiguo di somma minima e sottraiamo tale valore dalla somma totale di tutti gli elementi (*tot*).

```

int circularMaxSum(int[] A, int n)
{
    int maxSoFar = 0
    int maxHere = 0
    int minSoFar = 0
    int minHere = 0
    int tot = 0
    for i = 1 to n do
        minHere = min(minHere + A[i], 0)
        minSoFar = min(minSoFar, minHere)
        maxHere = max(maxHere + A[i], 0)
        maxSoFar = max(maxSoFar, maxHere)
        tot = tot + A[i]
    return max(maxSoFar, tot - minSoFar)
}

```

La procedura scorre gli n elementi di A una sola volta e utilizza operazioni costanti per ogni elemento, quindi la complessità temporale è $\Theta(n)$.

Si noti i due casi estremi in cui abbiamo solo numeri positivi o solo numeri negativi:

- Solo numeri positivi: la somma massimale corrisponde alla somma totale del vettore, la somma minimale è uguale a zero, il massimo fra somma massimale e somma totale - somma massimale è uguale alla somma totale, corretto;
- Solo numeri negativi: la somma massimale è uguale a zero, la somma minimale corrisponde alla somma totale del vettore, il massimo fra somma massimale e somma totale - somma minimale è uguale a zero, corretto.

Soluzioni errate

- Una soluzione errata consiste nel raddoppiare il vettore (copiandolo due volte in un vettore di appoggio), per poi applicare l'algoritmo di Kadane su di esso. È facile rendersi conto che è sbagliato, perché nel caso di un vettore di soli positivi viene restituito il doppio della loro somma.
- Qualcuno si è accorto di questo caso particolare e ha inserito un controllo specifico. Raramente un algoritmo "*che funziona quasi sempre, tranne in questo caso particolare qui*" è veramente corretto. Infatti, con $[2, -1, 2]$ non funziona: l'algoritmo di kadane prende l'intero sottovettore raddoppiato e restituisce 6, non 4 che è il valore corretto.
- Un'altra versione sbagliata: partendo dal valore *maxHere* nell'ultima posizione, si prova a estenderlo ricominciando dall'inizio. Di nuovo $[2, -1, 2]$ non va bene, perché *maxHere* è pari a $3=2-1+2$ e estendendolo con il 2 si ottiene 5. In pratica si è considerato il vettore $[2, -1, 2, 2]$ (dove l'ultimo 2 è il primo ripreso).

Esercizio B1 – Numeri irripetibili – Punti ≥ 8

L'algoritmo proposto utilizza una tecnica di *backtracking* per generare e stampare tutti i numeri compresi tra 0 (incluso) e 10^n (escluso) in cui nessuna cifra è ripetuta. La struttura del programma si basa su una funzione principale `printNumbers()` e una funzione ricorsiva ausiliaria `pnRec()`.

La funzione `printNumbers()` inizializza due strutture principali: un array booleano *set*, che tiene traccia delle cifre già utilizzate, e un array *S*, che rappresenta il numero in costruzione. Una cosa che hanno notato in pochi è che un numero con 11 cifre senza cifre ripetute è impossibile, quindi la dimensione massima dell'array è limitata a 10. L'array *set* è inizializzato con valori **true** per indicare che tutte le cifre da 0 a 9 sono disponibili; tuttavia poi fa un ciclo da 1 a 9 inserendo tale cifra in prima posizione e segnandola come "presa", per evitare numeri che iniziano con 0; la funzione ricorsiva `pnRec()` viene quindi chiamata con $i = 2$, indicando che si inizierà a costruire il numero dalla seconda posizione. Prima di questo, viene anche stampato il valore 0 per evitare cifre ripetute e per evitare.

La funzione `pnRec()` esplora tutte le cifre disponibili per la posizione corrente i del numero. Per ciascuna cifra k , viene verificato se questa è ancora disponibile, controllando il valore corrispondente in *set*[k]. Se la cifra è disponibile, viene assegnata alla posizione corrente *S*[i] e il numero parziale *S*[1... i] viene stampato. Successivamente, la cifra viene marcata come utilizzata (*set*[k] = **false**) e la funzione ricorsiva viene chiamata per generare il resto del numero. Al termine della chiamata ricorsiva, la cifra viene rimossa dalla soluzione corrente tramite un processo di *backtracking*, ripristinando la sua disponibilità (*set*[k] = **true**).

Questo approccio assicura che ogni numero generato soddisfi il vincolo di non avere cifre ripetute. L'esplorazione di tutte le cifre in ogni posizione garantisce che siano generati tutti i numeri validi con lunghezza massima n . Per valori di n inferiori, l'algoritmo stampa tutti i numeri con meno cifre, poiché la funzione stampa il numero anche prima di raggiungere la lunghezza n .

L'algoritmo è corretto perché ogni cifra è aggiunta solo se disponibile, evitando ripetizioni, e ogni combinazione di cifre viene esplorata esattamente una volta grazie alla natura del *backtracking*.

```
printNumbers(int n)
```

```
boolean[] set = new boolean[0...9] = {true}
int[] S = new boolean[1...min(10,n)]
print 0
for k = 1 to 9 do
    S[1] = k
    set[k] = false; pnRec(n, S, 2, set)
    set[k] = true;
```
